AI Proved the Code Memory Safe. Or Did It?

A critical audit of Anthropic and Theorem's memory safety proofs

Umang Mathur Prateek Saxena Tianyu Li Bo Wang Diego Meyer Ruishi Li
National University of Singapore
umathur@nus.edu.sg dcsprs@nus.edu.sg

1 November 2026

Abstract. Anthropic and Theorem recently announced machine-checked proofs about programs built from 123 open-source libraries [1]. Their proof starts with a specific setup of machine OS, processor, and test harness. Among other things, the announcement suggests that these proofs establish the absence of memory safety bugs under certain assumptions. However, to understand what a proof guarantees, it is imperative to examine the precise safety specification it establishes and ask whether it captures what we mean by memory safety. Should we believe the code proven safe under their setup is really so? We explain the gap between memory safety—respecting object bounds and lifetimes—and what the proof guarantees. Our review also identifies missing safety requirements in some published proof conditions. Closing this gap requires specifications that capture these requirements, including the contracts governing custom allocation and reuse. Developing and validating such specifications remains a central challenge.

Audience and purpose. This article is intended for a general audience. Computer-security experts will recognize the sanitizer limitations we discuss, and formal-verification experts will recognize the broader lesson: a proof shows that a program meets the requirements we have written down. We still need to ask whether those requirements capture what we mean by safety. As researchers, we believe it is important to make these distinctions accessible beyond our own communities, especially as advances in AI-assisted verification attract wider public attention. Through concrete examples from the published artifacts, we explain what the proofs establish, how their guarantees relate to familiar expectations of software safety, and where gaps remain. Our aim is to help readers understand both the significance of this achievement and the work still needed to obtain the guarantees they might reasonably expect.

1. The verification claim and our contention

The announcement. On 22 October 2026, researchers at Anthropic and Theorem announced verification results for programs built from 123 open-source libraries. The team used Claude to generate proofs about compiled programs and Rocq to check them. This is substantial evidence that automated verification with AI can scale. The team has released a subset of the proofs and the machine semantics, so others can scrutinize the guarantees [1].

The memory-safety claim. The blog initially presents its vision as an effort to eliminate "all bugs" using formal verification driven by AI, and in particular, the root causes of memory safety bugs. Their proof mechanises and scales the idea behind OSS-Fuzz, a popular testing service for open-source software [2]:

> "OSS-Fuzz automatically instruments C and C++ programs with AddressSanitizer (ASan), causing them to crash *whenever memory is accessed improperly*." [1]

The subtle but significant leap is *to treat ASan's checks as a complete specification of memory safety*. Proving that these checks never fail does not, by itself, rule out the memory errors ASan cannot detect.

This gap can remain even when a proof has been checked by Rocq [3]. The checker verifies that the proof establishes the theorem as stated, under its assumptions. If the theorem omits a memory-safety requirement, checking its proof does not fill in that requirement. To understand what the team has guaranteed, we therefore need to examine what their theorems actually say.

What the proofs establish. The team began with fuzzing harnesses; these are test programs that exercise selected library operations on inputs generated by a fuzzer. For each verified harness, they compared a plain binary with one instrumented with ASan, UndefinedBehaviorSanitizer (UBSan), or both [4, 5]. Under the stated execution-model and memory-budget assumptions, they proved that neither binary reaches a modeled error on any input covered by the theorem. They also proved that the two binaries have the same sets of observable behaviors. Sanitizer reports count as modeled errors and are therefore caught by the error-freedom guarantee. The team calls these properties *agreement statements* [5].

Assumptions we do not question. We accept the stated machine, operating-system, memory-budget, and test-harness assumptions. We also assume that the execution model, proof checker, and sanitizer are correctly implemented. We therefore accept the machine-checked proofs as mathematically valid.

Their caveats. A checked proof does not, by itself, connect the formal specification to the guarantee one desires. The announcement acknowledges this directly:

> "However, a proof checker can't tell you whether your spec matches reality." [1]

In their blog post, three gaps are acknowledged: differences between their model and deployed systems, incomplete coverage of library uses, and memory errors that ASan may miss.

Our contention. Memory safety bugs can let attackers steal data or take control of systems. CISA, the NSA, and other national cybersecurity agencies have urged software manufacturers to eliminate this class of vulnerability [6]. Unfortunately, much of the blog's messaging underplays the gap between what the proofs establish and a guarantee of memory safety. Even under the stated assumptions, the agreement theorems can hold while memory-safety bugs remain uncaught.

2. Why memory safety is not proved

Memory safety requires us to respect the boundaries and lifetimes of individual objects [7]. The checks used by these proofs do not capture all of these requirements. In particular, a program can access the wrong object or use an object after its lifetime has ended. This can happen **while** accessing memory that is still available to the program, without triggering an ASan error. Further, the original program and the version with ASan checks can agree on the same wrong behavior. Thus, the high-level theorem can still admit a correct proof. But it is important to note that we have then proved the requirements stated in the theorem. This does not, in itself, establish the absence of memory-safety bugs.

When we run a test, we use ASan's checks to tell us whether it has encountered a memory error. These checks are our *test oracle*. They are useful for finding bugs, even though they miss some. However, proving that these checks never fail does not rule out the memory errors they miss.

2.1 An overflow between fields

Consider this simplified interpreter fragment. An opcode selects a function from `handlers`. All 256 entries contain valid function addresses; entry `0xff` selects `h_admin`, and the others select `h_nop`.

```
typedef void (*handler)(void);
const handler handlers[256] = { ... };
struct S { char buf[20]; unsigned char op; };

static void fill(char *dst,
                 const uint8_t *in,
                 size_t n) {
    // BUG: should be i < 20
    for (size_t i = 0; i < n && i <= 20; ++i)
        dst[i] = (char)in[i];
}

struct S *x = malloc(sizeof *x);
if (x == NULL)
    return 0;
x->op = 0;
fill(x->buf, input, n);
handlers[x->op]();
free(x);
```

The violated boundary. In the checked builds, `op` immediately follows `buf`. With at least 21 input bytes, the loop writes beyond `buf[19]` and overwrites `op`. Writing `op` directly would be valid. The error is reaching it by copying beyond the bounds of `buf`. The addressed byte remains inside the enclosing allocation.

Why the checks accept it. Their execution model, Linux Process Semantics (LPS), permits the write because the allocation is writable. The ASan build places no poisoned boundary between these fields, so its check also accepts the write [4]. Moreover, `op` is an 8-bit value in these builds: every value selects a valid table entry. Changing it can select an unintended handler without sending execution to forbidden memory.

A checked guarantee. We proved the agreement statement for a reduced version in which both handlers are empty. Under the theorem's model and memory-budget assumptions, the fixed plain and ASan binaries avoid modeled errors and have the same observable behaviors. This guarantee does not require the copy to respect the boundary of `buf`. Appendix A gives the proof and execution-evidence details.

Security significance. The overflow is the vulnerability; changing the selected handler is a consequence. In an application with meaningful handlers, such corruption can change what the program does. Related attacks against Python and Lua illustrate this risk [8].

Here, the attacker changes data that selects valid code. **Ruling out some exploits does not establish the absence of the underlying vulnerability.**

2.2 The same issue in custom allocators

Suppose a developer obtains a 100-byte backing block through `malloc` and manages smaller objects within it using a pool. Two 50-byte objects are allocated through the following C-like interface:

```
char *p = pool_alloc(A, 50);
char *q = pool_alloc(A, 50);  // q == p + 50
```

Assume both allocations succeed. The first object is associated with the pointer `p`, and the second with `q`.

If we write `p[50] = 1`, we access memory beyond the first 50-byte object. In fact, we write into the first byte of the second object. However, this address still lies inside the 100-byte backing allocation. Without annotations describing the pool's internal boundaries, ASan can still mark it accessible. The out-of-bounds write isn't flagged.

The same distinction matters for object lifetimes. Suppose the pool releases the first object but retains its backing storage. Under the pool's contract, we must stop using the old reference `p`. Yet ASan may still mark those bytes accessible. Thus, a reference can have expired even though the memory it points to remains available.

The absence of a sanitizer report therefore does not mean the program is memory safe in a strict sense.

Appendix B gives the full example, including accesses after release and reuse of the same storage.

We next examine the published proofs. In particular, we ask whether their internal conditions enforce the object bounds and lifetime requirements. The agreement statement alone does not guarantee that these requirements hold.

3. Gaps in the proof contracts and coverage

Our initial investigation has identified several gaps in the published verification. In particular, some proof conditions do not establish the object bounds or lifetimes needed for memory safety. Further, the verified test harnesses leave out certain uses of the libraries that matter for these guarantees. We explain these findings below.

3.1 OpenSSL: a missing lifetime requirement

The finding. OpenSSL is a widely used cryptography library on which much of the internet's secure communication relies [9, 10]. Researchers have historically found serious memory-safety vulnerabilities in it. Heartbleed exposed process memory through a missing bounds check; two vulnerabilities reported in 2022 caused buffer overflows during certificate verification [11, 12].

Our finding concerns a different issue. In one of the team's published proofs for OpenSSL, we found a missing lifetime requirement. In particular, the condition used to decide whether a reference is valid can remain true even when that reference must no longer be used.

Temporary objects. OpenSSL uses a `BN_CTX` pool for temporary large integers. Cryptographic calculations need many such temporary objects. To avoid repeatedly allocating and freeing their storage, OpenSSL keeps them in a pool. A developer begins a frame, obtains temporaries from the pool, and uses them during a calculation. Here, a frame groups together temporaries whose lifetimes end together. The call `BN_CTX_end` ends the lifetime of the temporaries in a frame, although their backing storage can remain allocated [13, 14]. After this call, the old references must no longer be used.

The proof condition. The `bndiv` proof uses a condition called `valid_ref` to decide whether a reference to a temporary is valid. Here is its definition from the published `BnInv.v`, with only line breaks and indentation changed:

```
Definition valid_ref
    (r : bnref) (s : LS) : Prop :=
  match r with
  RPool i =>
      (i < 3)%nat /\
      cx_vals (ls_ctx s) <> []
  | _ => True
  end.
```

Here, `RPool i` denotes a reference to pool slot `i`. The condition requires `i` to be less than three and the list of pool values, `cx_vals`, to be nonempty. However, **it does not require the reference to belong to an active frame**. In other words, it does not require the temporary's lifetime to have remained active.

The missing guarantee. After `BN_CTX_end`, those pool values are retained in the model. We can therefore still establish `valid_ref` for a reference whose API-defined lifetime has ended. Thus, these local proof conditions can classify an expired reference as valid and do not enforce the temporary's lifetime. The pool storage still exists, but the reference has expired.

We independently checked this gap in Rocq. In a small client built from the published model, we can change a temporary through an expired reference while satisfying every condition that the proof requires for that operation [15].

Scope. In the verified `bndiv` client, the temporaries are used before their frame ends. We have not demonstrated an invalid access by this client. What we have identified is a missing lifetime requirement in the condition used to establish reference validity. To obtain the broader memory-safety guarantee, we need to express this requirement and prove that callers respect it.

Our review used Claude Code to inspect the source code and proof conditions. Appendix D gives the exact definitions and proof locations. It also discusses the stronger lifetime conditions present in the separate `bignum` proof.

3.2 coturn: a missing field bound

`coturn` is server software used to support internet calls. It implements two standard internet protocols, TURN and STUN [16, 17].

Buffer overflows have previously caused security vulnerabilities in `coturn`. In one reported case, a missing length check allowed data copied into an array to overwrite neighboring fields of the same structure [18].

Our investigation concerns a different write. We examined the part of the published proof that concerns writing into a message buffer. To establish memory safety here, we need to show that the write stays within the buffer. However, the proof conditions we inspected do not even rule out a write starting just before the buffer.

The following excerpts show its message-buffer layout and a write in its message-building code [19].[1] We have expanded the array sizes and marked omitted code.

```
typedef struct _stun_buffer {
  uint8_t channel[4];
  uint8_t buf[65507];
  /* other fields omitted */
} stun_buffer;

/* In stun_attr_add_str: */
const int clen =
```

[1] See `stun_buffer.h:45–51`, `stun_buffer.c:181–184`, and `ns_turn_msg.c:1666–1703`. The two-byte write helper is in `ns_turn_defs.h:145–148`.

```
  stun_get_command_message_len_str(buf, *len);
/* length calculation and checks omitted */
uint8_t *attr_start = buf + clen;
/* message-length updates omitted */
turn_write_u16(attr_start, attr);
```

Observe that `channel` and `buf` are fields of the same structure. In the reviewed call context, the `buf` argument points to the array field of a `stun_buffer`. The write helper stores two bytes. Its destination must be at or above the start of `buf` (let us denote this by b). The address $b-1$ belongs to `channel[3]`: it is inside the struct but outside the buffer field. A write starting here would therefore cross a field boundary, even though it stays inside the enclosing structure.

We checked whether the internal invariants exclude this address. These invariants are conditions that the proof establishes at particular points in the program. To demonstrate the gap, we constructed memory and register values satisfying the complete original invariant at a selected program point before the write. In this constructed state, the expression for the write's destination evaluates to $b-1$, one byte before `buf`. We used Rocq to check this construction. Thus, we have a machine-checked demonstration that these conditions do not establish the required buffer bound. We obtained this result for both the ASan and ASan+UBSan proofs [20]. Appendix E gives the exact proof locations.

In particular, these proof conditions admit an out-of-bounds destination. They do not correctly establish the buffer bound required for memory safety.

3.3 Other findings

Our review also found library uses that the published proofs do not cover.

`pupnp`: node reuse. In `pupnp`, developers can return a node to a `FreeList` cache and later obtain the same storage again. However, the allocation lemma we inspected assumes that the cache is empty. It proves the branch that allocates fresh storage. Thus, this lemma does not cover reusing a node from the cache. In particular, it does not establish the lifetime requirements for that reuse.

`msgpack-c`: zone reset. In `msgpack-c`, developers can clear a zone and then reuse its storage. Clearing the zone ends the lifetimes of the objects in it. Consider a client that obtains a reference to an object in the zone, clears the zone, and then uses the old reference. The reviewed harness does not include this sequence. Thus, its proof does not establish memory safety for this use of the library.

`simdjson`: parser reuse. In `simdjson`, developers can parse another document with the same parser, invalidating references into the previous document. Consider a client that keeps such a reference, parses a second document, and then uses the old reference. None of the proved harnesses includes this sequence. The proofs therefore do not establish memory safety for this use of the parser.

We inspected the source code and proof conditions for these three projects with assistance from Claude Code. Appendix D gives the detailed evidence, the exact proof conditions, and the useful local properties that the authors do establish.

`bzip2`: harness coverage. Our `bzip2` review illustrates a different limitation. The proved harness first compresses data and then decompresses the result. Its proof establishes that this round trip returns the original data. It also includes bounds on writes to the selector array. However, at the reviewed revision `98e995d4`, the separate harness for decompressing externally supplied compressed data remained unproved [5]. In particular, proving safety when decompressing the compressor's own output does not establish safety when decompressing externally supplied data. Appendix F records the evidence.

4. Specifications and intent

Constructing machine-checked proofs for real software is a substantial achievement. However, any claims that these programs have been proved memory safe overstate the guarantees we examined. To support such claims, one must also justify that the specifications capture the intended memory-safety requirements.

Our investigation illustrates this gap. In `coturn`, we constructed states that satisfy the complete internal invariants while violating an array-field bound. In OpenSSL's `bndiv` proof, the internal condition `valid_ref` can still hold for a pool reference after `BN_CTX_end` has ended its API-defined lifetime. Our other allocator reviews identified sequences of release and reuse outside the proofs' coverage. These findings identify missing conditions and limits on coverage; they do not, by themselves, establish reachable bugs in the reviewed programs.

Identifying the object boundaries and lifetimes that a specification should enforce is *itself a central challenge*. These requirements depend on how the program is intended to be used, and that intent may be only partly recoverable from the C code or from documentation. Some requirements become apparent only after a failure or an unexpected use exposes an omission.

Developing and validating specifications can therefore require domain expertise, discussions with developers and users, and careful review. AI tools may assist this process, but success in generating proofs does not establish that the requirements being proved adequately capture the intended behavior. When those requirements are unclear, deciding what ought to be proved remains a substantial problem. How should human judgment and automated methods contribute to identifying and validating such specifications?

References

[1] Anthropic. LLMs are making formal software verification feasible at a much larger scale, 2026. 22 October.

[2] Google. OSS-Fuzz documentation. Accessed 1 November 2026.

[3] The Rocq Prover. Rocq Reference Manual: Proof mode. Version 9.1.0. Accessed 1 November 2026.

[4] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov. AddressSanitizer: A Fast Address Sanity Checker. In *USENIX ATC*, pages 309–318, 2012.

[5] Theorem. OSS Verify: Initial public proof corpus, 2026. Audit snapshot `98e995d4`, 23 October.

[6] National Security Agency. U.S. and International Partners Issue Recommendations to Secure Software Products Through Memory Safety, 2023. 6 December.

[7] László Szekeres, Mathias Payer, Tao Wei, and Dawn Song. SoK: Eternal War in Memory. In *IEEE Symposium on Security and Privacy*, pages 48–62, 2013.

[8] Taemin Park, Julian Lettner, Yeoul Na, Stijn Volckaert, and Michael Franz. Bytecode corruption attacks are real—and how to defend against them. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*. Springer, 2018.

[9] OpenSSL Project. OpenSSL Library. Accessed 29 October 2026.

[10] OpenSSL Corporation. OpenSSL Corporation: Securing what comes next. Accessed 1 November 2026.

[11] OpenSSL Project. TLS heartbeat read overrun (CVE-2014-0160). Security advisory, 2014. 7 April.

[12] OpenSSL Project. X.509 email address buffer overflows (CVE-2022-3602 and CVE-2022-3786). Security advisory, 2022. 1 November.

[13] OpenSSL Project. `BN_CTX_start`. API documentation. Accessed 29 October 2026.

[14] OpenSSL Project. `bn_ctx` implementation. Pinned source revision `47aae188`.

[15] OpenSSL lifetime-contract audit: independent Rocq replay. Companion artifact, 2026. 1 November. Checked constructions, source hashes, and build records in the openssl-lifetime dossier.

[16] Internet Engineering Task Force. Traversal Using Relays around NAT (TURN): Relay Extensions to Session Traversal Utilities for NAT (STUN). RFC 8656, 2020. February.

[17] Internet Engineering Task Force. Session Traversal Utilities for NAT (STUN). RFC 8489, 2020. February.

[18] `coturn` Project. Stack buffer overflow in `decode_oauth_token_gcm()`. GitHub security advisory GHSA-74pg-rfh2-5qw5, CVE-2026-43994, 2026. 24 May. Fixed in version 4.10.0.

[19] `coturn` Project. `coturn` source. Revision `70ca74e0`, pinned by the reviewed proof build; accessed 1 November 2026.

[20] `coturn` field-bound invariant audit, 2026. Companion artifact, 1 November. Sources, proof locations and build records in the `coturn-field-bounds` dossier.

[21] Tianyu Li, Ruishi Li, Bo Wang, Diego Meyer, Umang Mathur, and Prateek Saxena. A checked agreement proof for an intra-object overflow. Companion artifact, `https://github.com/focs-lab/memory-safety-proof-audit`, 2026. The checked-example dossier contains proof sources, binaries, checker logs, and execution traces.

[22] Junwha Hong, Wonil Jang, Mijung Kim, Lei Yu, Yonghwi Kwon, and Yuseok Jeon. CMASan: Custom Memory Allocator-aware Address Sanitizer. In *IEEE Symposium on Security and Privacy*, pages 740–757, 2025.

[23] Chanyoung Park and Hyungon Moon. Efficient Use-After-Free Prevention with Opportunistic Page-Level Sweeping. In *NDSS*, 2024. Table I and Section VI.G.

[24] Arthur Azevedo de Amorim, Cătălin Hriţcu, and Benjamin C. Pierce. The Meaning of Memory Safety. In *Principles of Security and Trust (POST)*, pages 79–105, 2018.

[25] Umang Mathur. Allocator case studies: Companion reports, 2026. Selected proof reviews of `msgpack-c`, `simdjson`, OpenSSL, and `pupnp` at proof revision `98e995d4`.

[26] Theorem. OSS Verify: `coturn/FuzzStunClient` proofs, 2026. Revision `a6839bb9`, release 2026.10.2; accessed 1 November.

A. Memory safety and the agreement guarantee

Memory safety. An access must stay within the bounds of its object and occur while the reference remains valid. These are the spatial and temporal requirements of memory safety [7]. Merely accessing memory available to the process does not establish either requirement. A write can cross from one field into another, or use an expired reference, while the addressed bytes remain accessible.

What their theorem guarantees. Under the stated model and memory-budget assumptions, an agreement theorem establishes that neither binary reaches an error on any admitted input and initial layout [5]. It also establishes that the plain and sanitizer binaries have the same sets of observable behaviors. Sanitizer reports count as errors. However, agreement does not establish that either binary performs the operation the application intended. Both can agree on the same wrong behavior.

With the execution model. The Linux-process model imposes memory-access permissions. A write to read-only memory or a jump to non-executable memory causes an error. But the overwrite in our interpreter example stays within writable memory. The write crosses the boundary of `buf` and changes `op`, yet it does not violate these process-level permissions.

With ASan. ASan places poisoned redzones around allocations and delays reuse of freed storage. An instrumented access that triggers a sanitizer check is excluded by the theorem. But crossing an object boundary need not trigger that check. An access can skip a redzone and land in another live allocation. A stale reference can also reach accessible memory after reuse. These limitations are documented in the original ASan paper [4].

In our example, there is no poisoned boundary between `buf` and `op`. The overwrite therefore produces no ASan report. The same issue arises when custom allocators manage internal objects without annotations exposing their boundaries and lifetime changes. Appendix B develops that case.

Exploits versus vulnerabilities. Suppose an attacker can choose an indirect-call target freely. If an admitted input can send execution to forbidden memory, the agreement theorem cannot hold. This rules out that capability under the theorem's assumptions.

But an attacker may only need to select an unintended function at a valid executable address. In our example, every opcode selects a valid table entry. Both binaries can call the same unintended handler without a modeled error. **Ruling out some exploits does not establish the absence of the underlying vulnerability.**

A checked example. We proved the published `asan_agrees` statement for a reduced version of this example using OSS Verify at revision `a6839bb9`. The release checker accepted the proof for the fixed plain and ASan binaries. The retained execution traces confirm that the twenty-first input byte overwrites `op` and selects `h_admin` in both binaries. Both handlers are empty, so the observed consequence is unintended handler selection, not a privileged operation. The companion artifact contains the proof, binaries, checker logs, and execution traces [21].

We thus have a concrete program that violates an array boundary during execution. The agreement theorem still holds. To establish memory safety, we need additional requirements connecting each access to the bounds and lifetime of its intended object.

B. Custom allocators: bounds, release, and reuse

Setup. Suppose a pool obtains a 100-byte block through `malloc` and provides two adjacent 50-byte objects:

```
1 char *p = pool_alloc(A, 50);
2 char *q = pool_alloc(A, 50);  // q == p + 50
```

Assume both allocations succeed. The backing block remains allocated, and the pool makes no ASan poisoning updates. Under its contract, each reference permits access only within its object and expires on release. Reusing the storage does not revive an expired reference. Figure 1 shows three independent violations.

Bounds. The write `p[50] = 1` crosses the first object's boundary and changes the first byte of the second. ASan sees an accessible address and reports no error. A valid write through `q[0]` reaches the same byte: address accessibility alone does not distinguish the two accesses.

Release. Suppose we call `pool_free(A, p)` and then write `p[0] = 1`. The reference has expired, but the backing allocation remains accessible to ASan. Poisoning the released slot could expose this access.

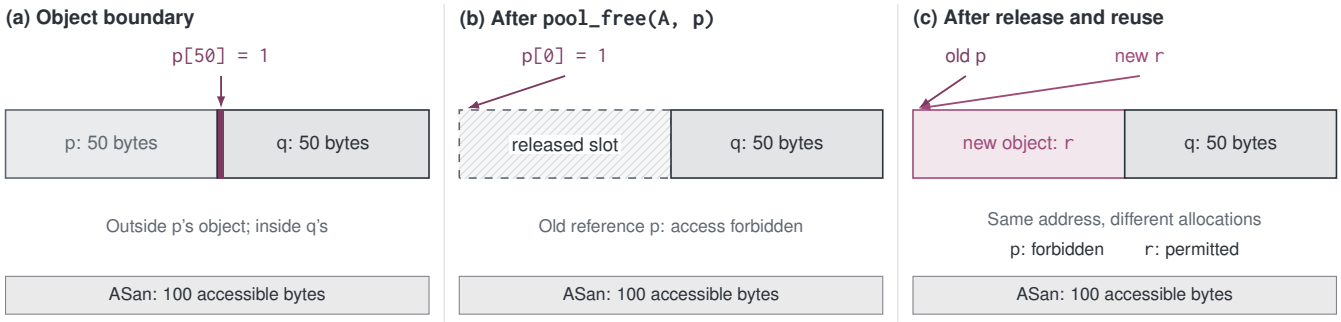


Figure 1: Three independent violations of the pool contract. All backing bytes remain accessible to ASan. After reuse, `r` permits access to the new object; `p` remains expired.

Reuse. In another execution, release `p` and allocate a new object through `r` at the same address. Access through `r` is permitted; access through the old `p` remains forbidden. Even if the slot was poisoned on release, making it accessible for `r` allows ASan's address check to accept both accesses. ASan's original paper documents this limitation after reuse [4].

Real examples. CMASan's evaluation found that ASan missed PHP's use-after-free vulnerability CVE-2015-2787 when the testing shim was disabled [22, Table 4]. This shim replaces custom allocation with standard allocation. Separately, HushVac evaluated an arbitrary-code-execution exploit for that vulnerability [23, Table I]. These studies establish detection failure and exploitability in different configurations.

All three accesses violate the pool's bounds or lifetime requirements. Sanitizer silence does not establish that these requirements hold. Appendix C describes how to state them explicitly.

C. Specifying logical memory safety

Scope. We state the bounds and lifetime obligations needed for our examples. Fix an instrumented binary and the executions covered by the verification assumptions. We write $\mathsf{MSafe}_{\mathrm{ASan}}$ when all ASan memory checks are satisfied in these executions, and $\mathsf{MSafe}_{\mathrm{logical}}$ when every access respects the intended object bounds, lifetimes, and operation-specific permissions. From $\mathsf{MSafe}_{\mathrm{ASan}}$ alone, we cannot conclude $\mathsf{MSafe}_{\mathrm{logical}}$. The agreement statements also establish freedom from other modeled errors and agreement of observable behavior.

Object identity. We must associate each access with the logical object or subobject that the reference and operation permit it to access. For example, a write through `buf[i]` must respect the bounds of `buf`, even when its address lies inside the enclosing struct. After pool storage is reused, an old reference still belongs to the earlier allocation. Choosing any live object containing the address would lose both distinctions.

Azevedo de Amorim et al. distinguish allocation identities from concrete addresses [24]. We adapt this distinction to struct fields and pool objects. The proof must track the association between references and objects through pointer operations and relate it to machine accesses. This association can be maintained as *ghost state*—information used in the proof—without adding runtime tags.

Access requirements. Let o be the object or subobject associated with an access. At time t, let $b_o(t)$ and $n_o(t)$ be its base address and size in bytes, and let $\mathrm{live}(o,t)$ mean that its logical lifetime has not ended. An access of $k > 0$ bytes at address a must satisfy

$$\mathrm{live}(o,t) \quad \wedge \quad [a, a+k) \subseteq [b_o(t), b_o(t) + n_o(t)).$$

The whole access must lie within the object's current bounds, and the address arithmetic must not wrap. Read or write permissions are separate requirements.

In Section 2.1, the write to `buf[20]` exceeds `buf`'s 20-byte extent. In Appendix B, the write to `p[50]` exceeds the first pool object's 50-byte extent. After release, access through `p` fails the lifetime condition, even if a new object occupies the same bytes.

Allocator operations. We must specify how allocator operations change these obligations. Release and bulk reset end the designated object lifetimes; shrinking reduces the bounds. Reuse creates a new identity, and the contract must specify which references survive relocation. We must also establish the contract's ownership, alignment, and internal-access permissions, and any required separation of live objects at the same allocation layer. Parent pools and their child objects overlap by design.

```c
BN_CTX *ctx = BN_CTX_new();
BN_CTX_start(ctx);
BIGNUM *t = BN_CTX_get(ctx);
compute_using(t);
BN_CTX_end(ctx);  // t must no longer be used.
BN_CTX_free(ctx);
```

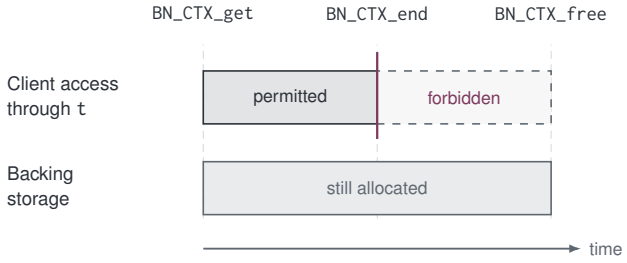


Figure 2: A `BN_CTX` temporary in one frame. Assume all allocations succeed. The developer must stop using `t` at `BN_CTX_end`, even though the backing storage can remain allocated until `BN_CTX_free` [13, 14].

D. Allocator contracts

Scope. The exported theorem does not tell us everything established within a proof. We therefore look at the internal conditions used to justify individual operations. In particular, we ask whether these conditions enforce the object bounds and lifetimes discussed in Appendix C. With assistance from Claude Code, we inspected selected proofs and harnesses for four projects at revision `98e995d4`.

OpenSSL. Consider a temporary obtained from a `BN_CTX` pool. We first begin a frame, obtain the temporary, and use it in a calculation. Calling `BN_CTX_end` ends its permitted lifetime [13]. The backing storage can remain allocated for later reuse [14]. Observe that retaining this storage does not make the old reference valid. Figure 2 shows this distinction.

Let us see how the `bndiv` proof represents reference validity. For a pool reference, its `valid_ref` condition requires the slot index to be less than three and the list of pool values to be nonempty. In particular, it does not require the temporary's frame to remain active. After `BN_CTX_end`, the model retains the pool values. Both tests can therefore still hold after the reference has expired.

A checked example. Satisfying `valid_ref` alone would not settle the question: another precondition might rule out the expired reference. We therefore checked the complete operation contract in Rocq 9.0, using the unchanged published definitions [15].

Starting from the model's initial state, our constructed client obtains a temporary and initializes it to one. It then ends the frame and changes the temporary's sign. Every precondition of `BN_set_negative_spec` holds, including state well-formedness and ownership. Thus, these local contracts permit an operation that violates the documented temporary lifetime.

We must distinguish this constructed client from the published `bndiv` client, which uses its temporaries before their frame ends. We have not demonstrated an invalid access by that client. The check concerns the modeled contracts; we did not replay the full binary agreement certificates.

We also found stronger lifetime reasoning in the separate `bignum` proof. It has an explicit liveness guard and preserves temporaries in active outer frames. This guard tracks active slots, although slot indices alone do not distinguish successive lifetimes when a slot is reused.

Other allocators. In `pupnp`, the allocation lemma we inspected assumes that the cache is empty. It therefore proves the branch that obtains fresh storage. We cannot use this lemma to justify taking a previously returned node from a populated cache. Other inspected contracts do represent cache return, track backing allocations, and require cached-node uniqueness.[2]

In `msgpack-c`, we found allocation-fit, alignment, and non-overlap reasoning. The harness unpacks an object, packs it, and then releases its storage. It does not clear a zone and continue using an old reference. Such a sequence would be a different client program.

[2] See `FreeListFree_post_A` and `FreeListFree_stmt_A`.

In simdjson, we found record-layout and decoder-bounds reasoning. The API invalidates a DOM string view when the same parser parses another document. None of the proved harnesses at this revision performs a second parse and then uses a retained old view. This invalid client use is therefore not covered by the proofs.[3]

What follows. A fixed client may respect lifetimes through its control flow and supporting invariants, even when an individual predicate does not express the full lifetime rule. To extend the guarantee to additional clients, we must state the lifetime requirements for their references. We must then prove that their accesses obey those requirements.

E. The coturn field-bound example

The question. We examine the write into buf discussed in Section 3. Let b be the start of this array field and p the write's destination. For the write to stay within buf, we need, in particular, $b \leq p$. We ask whether the selected proof invariants establish this bound.

The checked states. For both the ASan and ASan+UBSan builds, we construct a state satisfying the complete selected invariant with $p = b - 1$. Observe that this address belongs to channel[3]: it lies inside the surrounding structure, but outside buf. The byte at p is writable in the model, and its ASan shadow byte is zero.

Table 1 identifies the two cases. Here, s is the proof's initial stack anchor. We select case 1, using zero-based indexing, at each invariant location.

Table 1: The two checked coturn cases.

Build	Unit	Invariant location	Field start b	First byte-store
ASan	752	0x8d348	$s - 1326956$	0x8d38c
ASan+UBSan	845	0x8c93c	$s - 1327020$	0x8c97c

Proof references. We use proof revision a6839bb9 [26] and source revision 70ca74e0 [19]. The selected stores use stack-window and write-set checks that permit $b - 1$. Our constructions satisfy the full gamma2 predicate, using the original decoded certificates and unit parameters. They also satisfy the associated rank condition. The companion artifact's Complete.v contains both final existence theorems [20]. Rocq 9.2 checked these proofs; their reported assumptions are confined to upstream-allowed Rocq primitives.

What follows. These selected invariants do not imply the required field bound. The checked states occur before the store; we have not established an execution that reaches them or proceeds through the write. Thus, this result establishes neither a vulnerability in coturn nor a counterexample to the published agreement theorem.

F. The bzip2 proof and its coverage

We use proof-repository revision 98e995d4.

The round trip. The completed bzip2_compress_target proof includes a functional round-trip result. We compress an input and then decompress the resulting bytes. With a valid initial state and sufficient output space, the functional model returns the original input. The results rt_small and b2b_round_trip establish this property.

The field bound. We also examined the proof of the loop that fills selectorMtf, an array with 18,002 entries. Observe that the loop can process more selectors than the array holds. The code therefore checks the index before writing: when the index is too large, it skips the store. The proof follows both branches and combines the guard with the loop invariant to establish that each store stays within this array field. We therefore found the required field bound for this write.

The coverage limit. The completed harness passes the compressor's output to the decompressor. In particular, it does not supply arbitrary compressed bytes directly. The local selector-loop reasoning allows larger counts than the compressor produces. But this does not give us a completed proof for arbitrary-input decompression. The separate bzip2_decompress_target harness supplies input bytes directly; its agreement statements remain open at the reviewed revision. This is a limit of the public proof coverage. It does not establish a vulnerability in bzip2 or a flaw in the completed round-trip proof.

[3] The companion reports [25] record the msgpack-c allocation lemmas, the simdjson bounds reasoning, and the harness inventory. The pinned DOM API documentation states the string-view invalidation rule.